Experiment No. 2
Create a Blockchain using Python

Aim

To create a blockchain using Python.

Theory

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, a proof (from Proof-of-Work) and the hash of the previous block, which ensures integrity and security.

In this program:

1. **Block creation** – a block is generated using `create_block()` with a proof and the previous hash.
2. **Proof-of-Work (PoW)** – a block is mined by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – the SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block references the previous hash correctly and satisfies the PoW condition.
5. **Flask web API** – the routes `/mine_block`, `/get_chain` and `/is_valid` allow mining, fetching the chain and verifying its integrity through a browser or API client.

The program runs on a local server and simulates mining a blockchain without a network of nodes, making it an introductory model for understanding blockchain basics.

Requirements

- Python 3 and `flask==2.2.5` (`pip install flask==2.2.5`).
- Web browser (or an API client) to call the endpoints.

Procedure

Step 1: Write the program (blockchain.py)

```
# ------------------------------------------------------------
# Module 1 - Create a Simple Blockchain (Fully Commented)
# ------------------------------------------------------------
# Required installation:(run cmd as administrator)
# pip install flask==2.2.5


# Importing libraries
import datetime   # To generate timestamps for blocks
```

```
import hashlib    # To generate SHA256 hashes
import json   # To convert block data into JSON


from flask import Flask, jsonify   # To create API endpoints


# ------------------------------------------------------------
# Part 1 - Building the Blockchain
# ------------------------------------------------------------

class Blockchain:
    def __init__(self):
        # This list will store the entire chain of blocks
        self.chain = []

        # Create the genesis block (first block)
        # proof = 1 -> arbitrary
        # previous_hash = '0' -> since no previous block exists
        self.create_block(proof=1, previous_hash="0")

    def create_block(self, proof, previous_hash):
        """
        Creates a new block and adds it to the chain.
        Each block contains:
        - index
        - timestamp
        - proof (PoW output)
        - previous block hash
        """
        block = {
            "index": len(self.chain) + 1,    # Position of block in chain
            "timestamp": str(datetime.datetime.now()),    # Current timestamp
            "proof": proof,   # PoW result
            "previous_hash": previous_hash,     # Hash of the previous block
        }
        # Add block to the chain
        self.chain.append(block)
        return block

    def get_previous_block(self):
        """
        Returns the last block in the chain.
        """
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        """
        Simple Proof of Work (PoW) algorithm:
        - Find a number (new_proof)
        - Such that the SHA256 hash of (new_proof^2 - previous_proof^2)
          starts with '0000'

        This is a computational puzzle to secure the network.
        """
        new_proof = 1
        check_proof = False

        while check_proof is False:
            # Cryptographic puzzle: hash difference of squares
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()

            # Check if hash begins with 4 leading zeros
            if hash_operation[:4] == "0000":
                check_proof = True
            else:
                new_proof += 1

        return new_proof

    def hash(self, block):
```

```
        """
        Creates a SHA256 hash of a block.
        - Sort keys to ensure consistent hashing
        """
        encoded_block = json.dumps(block, sort_keys=True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        """
        Validates the blockchain by checking:
        1. The previous_hash field matches the actual hash of the previous block.
        2. The PoW condition (hash starting with '0000') is satisfied for each block.
        """
        previous_block = chain[0]      # Genesis block
        block_index = 1   # Start checking from block 2

        while block_index < len(chain):
            block = chain[block_index]

            # Check previous hash correctness
            if block["previous_hash"] != self.hash(previous_block):
                return False

            # Validate Proof of Work
            previous_proof = previous_block["proof"]
            proof = block["proof"]
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:4] != "0000":
                return False

            # Move to next block
            previous_block = block
            block_index += 1

        return True


# ------------------------------------------------------------
# Part 2 - Mining our Blockchain (Flask API)
# ------------------------------------------------------------

# Initialize Flask web application
app = Flask(__name__)

# Create an instance of the Blockchain
blockchain = Blockchain()


# ------------------------------------------------------------
# API Endpoint: Mine a new block
# ------------------------------------------------------------
@app.route("/mine_block", methods=["GET"])
def mine_block():
    # Get the previous block
    previous_block = blockchain.get_previous_block()

    # Extract previous proof
    previous_proof = previous_block["proof"]

    # Run Proof of Work algorithm
    proof = blockchain.proof_of_work(previous_proof)

    # Get hash of previous block
    previous_hash = blockchain.hash(previous_block)

    # Create the new block
    block = blockchain.create_block(proof, previous_hash)

    # Prepare response
    response = {
```

```
        "message": "Congratulations, you just mined a block!",
        "index": block["index"],
        "timestamp": block["timestamp"],
        "proof": block["proof"],
        "previous_hash": block["previous_hash"],
    }
    return jsonify(response), 200


# ---------------------------------------------------------
# API Endpoint: Get the full blockchain
# ---------------------------------------------------------
@app.route("/get_chain", methods=["GET"])
def get_chain():
    response = {"chain": blockchain.chain, "length": len(blockchain.chain)}
    return jsonify(response), 200


# ---------------------------------------------------------
# API Endpoint: Check if blockchain is valid
# ---------------------------------------------------------
@app.route("/is_valid", methods=["GET"])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)

    if is_valid:
        response = {"message": "All good. The Blockchain is valid."}
    else:
        response = {
            "message": "Houston, we have a problem. The Blockchain is not valid."
        }

    return jsonify(response), 200


# ---------------------------------------------------------
# Run the Flask app
# ---------------------------------------------------------
app.run(host="0.0.0.0", port=5000)
```

Step 2: Run the server

```
python blockchain.py
```

Step 3: Call the endpoints

Open `/get_chain`, `/mine_block` and `/is_valid` in the browser.

Output

Server log:

```
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.43.57:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Sep/2026 12:51:18] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [02/Sep/2026 12:51:18] "GET /favicon.ico HTTP/1.1" 404 -
192.168.43.57 - - [02/Sep/2026 12:51:25] "GET / HTTP/1.1" 404 -
192.168.43.57 - - [02/Sep/2026 12:51:25] "GET /favicon.ico HTTP/1.1" 404 -
192.168.43.57 - - [02/Sep/2026 12:51:33] "GET / HTTP/1.1" 404 -
192.168.43.57 - - [02/Sep/2026 12:51:49] "GET /get_chain HTTP/1.1" 200 -
```

```
192.168.43.57 - - [02/Sep/2026 12:52:10] "GET /mine_block HTTP/1.1" 200 -
192.168.43.57 - - [02/Sep/2026 12:52:20] "GET /is_valid HTTP/1.1" 200
```

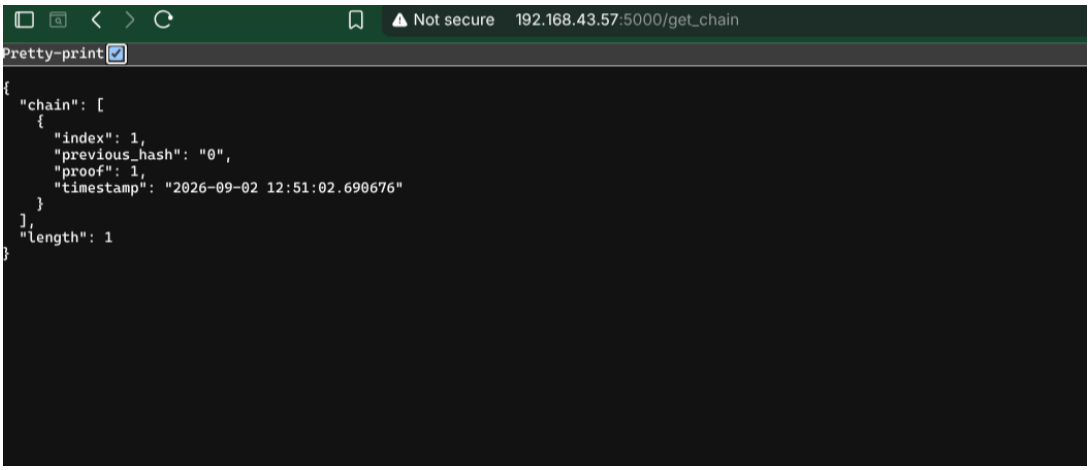


Figure 1: `/get_chain`: the chain contains only the genesis block

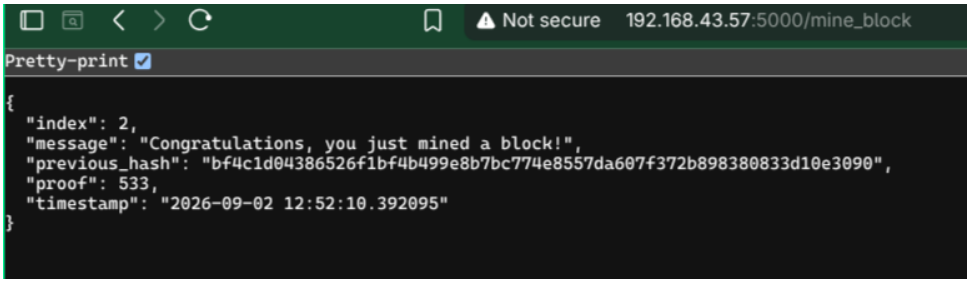


Figure 2: `/mine_block`: a new block is mined

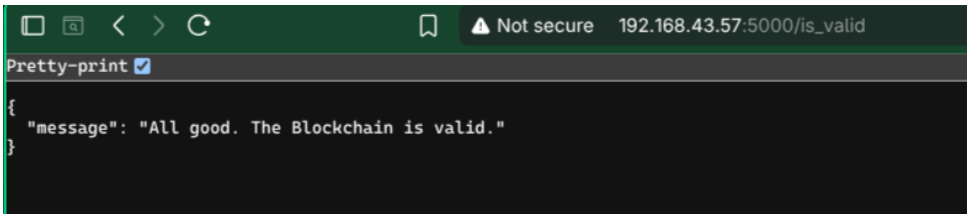


Figure 3: `/is_valid`: the blockchain is valid

Conclusion

A basic blockchain was created using Python and Flask, supporting block mining, chain retrieval and validity checks. Proof-of-Work secures the chain by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles (decentralization, immutability and consensus) in a simplified environment.

Experiment completed successfully.